UNIWERSYTET MARII CURIE-SKŁODOWSKIEJ
W LUBLINIE
Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Geoinformatyka**
Specjalność:

Michał Jan Wilkos
Nr albumu: 307370

Projekt i implementacja oprogramowania do analizy przestrzenno-czasowej danych treningowych z urządzeń GPS

Design and implementation of software for spatial-temporal analysis of training data from GPS devices

Praca magisterska
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki
pod kierunkiem dr Michała Klisowskiego

Lublin 2026

Spis treści

Wstęp **5**

1 Teoria (do poprawy) **7**

1.1 Pamięć (do poprawy) 7

1.1.1 Pamięć podręczna i hierarchia pamięci 7

1.1.2 Równoległe wykonywanie operacji 8

1.1.3 Wektoryzacja 9

1.2 GPS 10

1.2.1 Formaty danych GPS 11

1.2.2 GPX 11

1.2.3 TCX 11

1.2.4 FIT 14

Bibliografia **15**

Wstęp

W czasach rosnącej świadomości na temat zdrowego trybu życia oraz dynamicznego rozwoju technologii pomiaru sesji treningowych coraz więcej osób decyduje się na monitorowanie swoich aktywności fizycznych, a popularność zegarków sportowych i urządzeń z modułami GPS rośnie z roku na rok. Jednak generowanie ogromnych ilości danych o przebytych dystansach i parametrach treningowych niesie ze sobą problemy nie tylko w kontekście interpretacji, ale przede wszystkim obliczeń, szczególnie w przypadku przetwarzania informacji przestrzenno-czasowych.

W procesie tworzenia oprogramowania analitycznego ważną rolę odgrywa wybór odpowiedniej metody przetwarzania dużych zbiorów danych. Chociaż język Python oferuje mnogość narzędzi, wiele rozwiązań różni się pod względem efektywności czasowej i pamięciowej. Często stosowane standardowe struktury danych, takie jak listy, mogą okazać się niewystarczające przy skomplikowanych i obciążających pamięć obliczeniach. Te zagadnienia oraz chęć sprawdzenia różnic w wydajności stanowiły inspirację do stworzenia aplikacji, która posłuży jako środowisko testowe dla tych metod.

Praca będzie się koncentrować na opracowaniu oraz zaimplementowaniu oprogramowania do analizy przestrzenno-czasowej danych treningowych, z uwzględnieniem porównania efektywności różnych metod algorytmów analitycznych. Głównym założeniem jest nie tylko stworzenie funkcjonalnego narzędzia, ale także zbadanie i zestawienie wydajności operacji wykonywanych na podstawowych strukturach danych w porównaniu do rozwiązań wektorowych.

Struktura pracy zostanie podzielona na trzy rozdziały. Pierwszy z nich omówi wykorzystaną technologię oraz charakterystykę przetwarzania danych GPS, ze wskazaniem na różnice pomiędzy klasycznym modelem programowania a podejściem wektorowym. Drugi rozdział będzie przedstawiać założenia projektowe aplikacji oraz metodologię przeprowadzania testów porównawczych. W ostatnim rozdziale przedstawione zostaną aspekty implementacyjne, w tym organizacja kodu oraz napotkane problemy, a jego ważną część będzie stanowić prezentacja wyników analizy porównawczej wydajności zastosowanych rozwiązań w odniesieniu do czasu przetwarzania i wykorzystania zasobów systemowych.

Rozdział 1

Teoria (do poprawy)

Ten rozdział poruszy tematy dotyczące pamięci, podstaw przetwarzania danych, które stanowią bazę do zrozumienia mechanizmów optymalizacji oprogramowania, a także przybliży architektury systemów nawigacji satelitarnej oraz różne formaty przechowujące dane przestrzenne.

1.1 Pamięć (do poprawy)

1.1.1 Pamięć podręczna i hierarchia pamięci

Współczesne procesory wykonują obliczenia szybciej, niż system pamięci jest w stanie dostarczyć im dane. Jest to problem znany jako „wąskie gardło" występujący w architekturze von Neumanna, według której dane są przechowywane w pamięci wraz z instrukcjami kodu programu[1]. Problem ten określa się również jako ścianę pamięci (ang. *memory wall*). To sprawia, że w aplikacjach przetwarzających duże ilości danych problemem staje się opóźnienie w dostępie do pamięci RAM, a nie moc obliczeniowa rdzenia.

Aby złagodzić ten problem, architektury komputerowe wykorzystują wielopoziomową pamięć podręczną, która stanowi swoisty bufor pomiędzy procesorem a pamięcią RAM. Dzieli się ona na trzy poziomy, z czego pierwszy z nich jest najszybszy, a trzeci – najwolniejszy[1]. Jest to mniejsza pamięć, ale znacznie szybsza od pamięci operacyjnej komputera. W celu uzyskania wysokiej wydajności często zarządza się danymi tak, aby procesor jak najczęściej znajdował potrzebne informacje w pamięci podręcznej, ograniczając kosztowne pobrania z pamięci głównej RAM.

Fundamentem wydajnego wykorzystania pamięci podręcznej jest zasada lokalności odwołań:

- Lokalność czasowa: Dane, do których uzyskano dostęp, prawdopodobnie będą potrzebne ponownie w niedługim czasie.

- Lokalność przestrzenna: Jeśli program odwołuje się do określonego adresu pamięci, z dużym prawdopodobieństwem wkrótce odwoła się do sąsiednich adresów[1].

Z perspektywy programisty i wyboru struktur danych w projekcie, lokalność przestrzenna ma bardzo ważne znaczenie. Procesory nie pobierają z pamięci pojedynczych bajtów, lecz całe linie pamięci podręcznej. Struktury takie jak tablice dynamiczne, gdzie elementy są ułożone w pamięci w kolejności jeden za drugim, są pod tym względem bardzo wydajne. Pobranie pierwszego elementu automatycznie ładuje do cache'u kolejne, co pozwala na płynne dostarczanie danych[2].

Struktury oparte na węzłach i wskaźnikach, takie jak listy połączone lub drzewa charakteryzują się rozrzuceniem elementów po stercie pamięci. Przejście do kolejnego elementu wymaga skoku do losowego adresu, co często skutkuje chybieniem pamięci podręcznej i wstrzymaniem pracy procesora na czas pobrania danych z RAM[2].

Współczesne procesory wielordzeniowe zazwyczaj implementują trójpoziomowy system pamięci podręcznej. Pamięć podręczna pierwszego poziomu jest najszybszą i najmniejszą spośród wszystkich trzech poziomów. Dzieli się na dwie niezależne sekcje – L1i dla instrukcji oraz L1d dla danych. Taki podział zapobiega konfliktom między pobieraniem kodu a odczytem danych operacyjnych. Rozmiar tej pamięci waha się od 32 do 48 KB na rdzeń. [1]

Pamięć drugiego poziomu stanowi bufor między pamięcią L1 a wolniejszymi zasobami. Jej pojemność waha się od 256 KB do 2 MB na rdzeń. W przypadku „chybienia" w L1, procesor szuka danych w L2, co zapobiega wstrzymaniu pracy.[1]

Pamięć trzeciego poziomu jest największą pamięcią podręczną w procesorze. Zazwyczaj jest ona współdzielona przez wszystkie rdzenie w procesorze. Jej rozmiar może wynosić od kilkunastu do ponad stu megabajtów. Głównym zadaniem pamięci podręcznej trzeciego poziomu jest synchronizacja danych między rdzeniami, a także redukcja odwołań do pamięci RAM.[1]

Ważną kwestią do poruszenia w tym temacie jest mechanika transferu danych. System pamięci nie przesyła pojedynczych bajtów czy zmiennych. Komunikacja między pamięciami RAM, L3, L2 i L1 odbywa się za pomocą tzw. linii pamięci podręcznej (ang. *cache lines*). Oznacza to, że pojedyncze odwołanie do pamięci przez procesor powoduje pobranie całego 64-bajtowego bloku.[1]

1.1.2 Równoległe wykonywanie operacji

Przez dekady wzrost wydajności aplikacji opierał się na zwiększaniu częstotliwości taktowania procesorów, lecz skok technologiczny sprawił, że dalsze zwiększanie prędkości pojedynczego rdzenia stało się nieefektywne. Zamiast tego, zaczęto zwiększać liczbę rdzeni w procesorach, co doprowadziło do zmiany w projektowaniu systemów i aplikacji – przyspieszania pojedynczych instrukcji w stronę równoległych obliczeń[3].

Do klasyfikacji architektur równoległych stosuje się taksonomię zaproponowaną przez Michaela Flynna w 1966 roku

- SISD (ang. *Single Instruction, Single Data*): Architektura von Neumanna, gdzie jeden procesor wykonuje jedną instrukcję na jednej danej. Jest to model sekwencyjny.
- SIMD (ang. *Single Instruction, Multiple Data*): Jedna instrukcja sterująca wykonuje tę samą operację na wielu elementach danych jednocześnie. Jest to podstawa wektoryzacji.
- MISD (ang. *Multiple Instruction, Single Data*): Wiele instrukcji operuje na jednej danej.
- MIMD (ang. *Multiple Instruction, Multiple Data*): Wiele procesorów lub rdzeni wykonuje niezależnie różne instrukcje na różnych danych. Jest to podstawowy model równoległości[3].

Architektura MIMD jest realizowana w obecnych komputerach za pomocą procesorów wielordzeniowych. W kontekście analizy danych treningowych pozwala to na podział pracy na mniejsze, niezależne zadania. System może analizować kilka niezależnych plików jednocześnie, gdzie każdy plik obsługiwany jest przez osobny wątek procesora. W związku z tym system może działać w przykładowy sposób:

- Rdzeń 1 analizuje trasę biegu z poniedziałku.
- Rdzeń 2 analizuje trasę rowerową z wtorku.
- Rdzeń 3 analizuje spacer ze środy.

Ponieważ wynik analizy jednego działania nie zależy od wyniku drugiego, rdzenie te nie muszą się ze sobą komunikować ani na siebie czekać.[3]

1.1.3 Wektoryzacja

Według taksonomii zaproponowanej przez Michaela J. Flynna, tradycyjne jednostki obliczeniowe operują w modelu SISD. Oznacza to, że pojedyncza instrukcja maszynowa modyfikuje stan pojedynczego parametru w jednym cyklu maszynowym. Alternatywą, która zdominowała współczesne jednostki CPU, jest model SIMD. Paradygmat ten eksploatuje zjawisko równoległości na poziomie danych (ang. *Data-Level Parallelism*), umożliwiając zaaplikowanie tej samej operacji (np. dodawania, mnożenia) do wektora elementów rozlokowanych w dedykowanym, szerokim rejestrze sprzętowym.[3]

Zysk wydajnościowy płynący z wektoryzacji można opisać za pomocą prawa Amdahla. Jeżeli P oznacza ułamek programu podatny na zrównoleglenie wektorowe, a N

oznacza liczbę elementów przetwarzanych jednocześnie w jednym rejestrze wektorowym, to teoretyczne maksymalne przyspieszenie S (ang. *speedup*) określa wzór:

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Z powyższego równiania wynika, że skuteczność technologii SIMD jest ściśle zdeterminowana przez to, jak duża część algorytmu daje się wyrazić w postaci niezależnych operacji wektorowych.[3]

Sama translacja instrukcji na postać wektorową nie gwarantuje proporcjonalnego przyspieszenia. Optymalizacja SIMD napotyka na dwa fundamentalne wyzwania, które muszą zostać uwzględnione w projektowaniu algorytmów. Najczęstszym powodem takiej nieefektywności jest niewystarczająca przepustowość szyny pamięci pomiędzy RAM a pamięcią podręczną procesora. Procesor jest w stanie wykonywać operacje SIMD znacznie szybciej, niż system pamięci dostarcza nowe dane.[3]

Bezpośrednią konsekwencją specyfiki ładowania danych wektorowych jest konieczność zmiany systemów projektowania struktur danych w pamięci (ang. *Data-Oriented Design*). Klasyczne programowanie obiektowe naturalnie układa dane w sposób AoS, czyli tablice struktur (ang. *Array of Structures*). Przylegające do siebie w pamięci bajty reprezentują różne atrybuty pojedynczego obiektu (np. x, y, z, kolor, czas). Utrudnia to załadowanie np. wyłącznie współrzędnych x z kolejnych elementów. Dlatego też aplikacje zorientowane na wysoką wydajność obliczeniową korzystają z ułożenia SoA, czyli struktury tablic (ang. *Structure of Arrays*), w której atrybuty tego samego typu tworzą oddzielne, ciągłe obszary pamięci.[4]

Jeżeli algorytm odwołuje się do pamięci w sposób losowy lub nieliniowy, czyli tak jak ma to miejsce w przypadku układu tablic struktur, wczytana 64-bajtowa linia będzie zawierać wiele bezużytecznych w danym momencie informacji. To sprawia, że procesor szybko wykorzysta całą pojemność pamięci podrecznęj pierwszego poziomu.

Z drugiej strony, gdy dane są ułożone w pamięci sekwencyjnie (układ struktur tablic), pobranie pierwszego elementu spowoduje załadowanie kolejnych elementów do pamięci L1. Dzięki temu rozwiązaniu procesor, wykonując instrukcje wektorowe, znajdzie kolejne potrzebne dane już w najszybszej pamięci.[4]

1.2 GPS

Global Positioning System (GPS), oficjalnie znany jako NAVSTAR GPS, to satelitarny system nawigacyjny stworzony i zarządzany przez Departament Obrony Stanów Zjednoczonych. Należy on do szerszej rodziny systemów klasy GNSS (ang. *Global Navigation Satellite System*), do której zaliczają się również europejski system Galileo, rosyjski GLONASS oraz chiński BeiDou. Choć w mowie potocznej akronim "GPS" stosuje się

uniwersalnie do określenia każdego rodzaju nawigacji satelitarnej, w dosłownym ujęciu odnosi się on do konkretnego amerykańskiego systemu satelitarnego.

Zasada działania opiera się na pomiarze czasu propagacji sygnału. Jeżeli odbiornik wie, o której godzinie sygnał został wysłany przez satelitę i o której dotarł do anteny, może obliczyć czas przelotu (ang. *Time of Flight*). Odległość od satelity to iloczyn czasu przelotu i prędkości światła. Odległość ta determinuje miejsce w przestrzeni, gdzie znajduje się odbiornik.[5]

1.2.1 Formaty danych GPS

Wraz z popularyzacją systemów informacji geograficznej (GIS) i osobistych urządzeń nawigacyjnych, zaistniała potrzeba ustandaryzowania zapisu w sposób umożliwiający łatwą wymianę informacji między różnymi platformami. Doprowadziło to do powstania formatów opartych na języku znaczników XML (ang. *Extensible Markup Language*), z których najpowszechniejszym stał się GPX (ang. *GPS Exchange Format*). GPX zdefiniował podstawowy system opisu tras (ang. *Routes*), śladów (ang. *Tracks*) i punktów (Waypoints). Jednakże, wraz z rozwojem telemetrii sportowej i smartwatchy, nawigacyjny charakter formatu GPX okazał się niewystarczający, co wymusiło stworzenie nowych formatów, integrujących dane przestrzenne z fizjologicznymi.[6]

1.2.2 GPX

1.2.3 TCX

Format TCX (Training Center XML) został zaprojektowany i wprowadzony przez firmę Garmin w 2007 roku jako część oprogramowania Garmin Training Center. W przeciwieństwie do uniwersalnego formatu GPX, TCX został zbudowany od podstaw z myślą o analityce sportowej, rekreacji i monitorowaniu wysiłku fizycznego.

Z punktu widzenia inżynierii danych, plik TCX jest dokumentem XML podlegającym walidacji przez schemat XSD (XML Schema Definition), opublikowany w przestrzeni nazw http://www.garmin.com/xmlschemas/TrainingCenterDatabase/v2.[7]

Listing 1.1: Przykładowy plik TCX

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <TrainingCenterDatabase
3   xsi:schemaLocation="http://www.garmin.com/xmlschemas/
      TrainingCenterDatabase/v2 http://www.garmin.com/xmlschemas/
      TrainingCenterDatabasev2.xsd"
4   xmlns:ns5="http://www.garmin.com/xmlschemas/ActivityGoals/v1"
```

```xml
  xmlns:ns3="http://www.garmin.com/xmlschemas/ActivityExtension/v2"
  xmlns:ns2="http://www.garmin.com/xmlschemas/UserProfile/v2"
  xmlns="http://www.garmin.com/xmlschemas/TrainingCenterDatabase/v2"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ns4="http://www.garmin.com/xmlschemas/ProfileExtension/v1">
  <Activities>
    <Activity Sport="Running">
      <Id>2025-05-06T04:58:19.000Z</Id>
      <Lap StartTime="2025-05-06T04:58:19.000Z">
        <TotalTimeSeconds>335.479</TotalTimeSeconds>
        <DistanceMeters>1000.0</DistanceMeters>
        <MaximumSpeed>3.4149999618530273</MaximumSpeed>
        <Calories>58</Calories>
        <AverageHeartRateBpm>
          <Value>146</Value>
        </AverageHeartRateBpm>
        <MaximumHeartRateBpm>
          <Value>158</Value>
        </MaximumHeartRateBpm>
        <Intensity>Active</Intensity>
        <TriggerMethod>Manual</TriggerMethod>
        <Track>
          <Trackpoint>
            <Time>2025-05-06T04:58:19.000Z</Time>
            <Position>
              <LatitudeDegrees>51.23510423116386</LatitudeDegrees>
              <LongitudeDegrees>22.55628313869238</LongitudeDegrees>
            </Position>
            <AltitudeMeters>171.39999389648438</AltitudeMeters>
            <DistanceMeters>0.8999999761581421</DistanceMeters>
            <HeartRateBpm>
              <Value>113</Value>
            </HeartRateBpm>
```

```
37                <Extensions>
38                  <ns3:TPX>
39                    <ns3:Speed>0.8679999709129333</ns3:Speed>
40                    <ns3:RunCadence>0</ns3:RunCadence>
41                  </ns3:TPX>
42                </Extensions>
43              </Trackpoint>
44              <Trackpoint>
45            </Track>
46          </Lap>
47        </Activity>
48      </Activities>
49    </TrainingCenterDatabase>
```

Główną cechą odróżniającą TCX od innych formatów jest jego orientacja na "aktywność" (ang. *Activity*), a nie tylko i wyłącznie na "ścieżkę" (ang. *Track*). Plik TCX modeluje zbiór danych telemetrycznych w oparciu o zagnieżdżoną hierarchię:

- TrainingCenterDatabase: Główny węzeł dokumentu, który zawiera w sobie aktywności użytkownika.
- Activities / Activity: Reprezentuje pojedynczy trening. Atrybut Sport węzła Activity natychmiast kategoryzuje rodzaj ruchu (np. Running, Biking).
- Lap (Okrążenie): Jednostka podziału treningu w TCX. TCX dzieli aktywność na logiczne okrążenia (definiowane ręcznie przez użytkownika za pomocą przycisku na zegarku lub automatycznie, np. co 1 kilometr). Węzeł Lap przechowuje od razu zagregowane statystyki: TotalTimeSeconds (całkowity czas), DistanceMeters (dystans), MaximumSpeed (prędkość maksymalna) oraz Calories (spalone kalorie) itp.
- Track / Trackpoint: Wewnątrz każdego okrążenia znajduje się ścieżka, stanowiąca zbiór punktów pomiarowych (Trackpoint).[7]

Pojedynczy punkt pomiarowy w formacie TCX stanowi połączenie danych z różnych sensorów. Podczas gdy tradycyjny odbiornik GPS dostarcza głównie współrzędnych i czasu, zegarek sportowy synchronizuje w tym samym momencie dane z pulsometru, barometru i akcelerometru.

Struktura informacyjna węzła Trackpoint obejmuje:

- <Time>: Znacznik czasu w formacie ISO 8601 (np. 2023-05-14T08:30:00.000Z).

- <Position>: Blok przestrzenny zawierający <LatitudeDegrees> i <LongitudeDegrees>. Reprezentacja bazuje na układzie WGS 84 w formie stopni dziesiętnych. Element ten jest opcjonalny – np. w przypadku biegu na bieżni mechanicznej, zegarek generuje punkty TCX pozbawione bloku ¡Position¿, ale zawierające tętno i czas.
- <AltitudeMeters>: Wysokość, najczęściej z dokładnego czujnika barometrycznego kalibrowanego sygnałem GPS.
- <DistanceMeters>: Skumulowany dystans od początku okrążenia.
- <HeartRateBpm>: Informacja o tętnie w uderzeniach na minutę.[7]

Z uwagi na fakt, że oryginalny schemat TCX powstał kilkanaście lat temu, nie przewidywał on nowoczesnych metryk, takich jak moc generowana podczas biegu, balans stóp czy saturacja krwi. Aby uniknąć konieczności ciągłego modyfikowania głównego schematu, Garmin wprowadził mechanizm rozszerzeń w oparciu o nowe przestrzenie nazw (ActivityExtension/v2). Wewnątrz węzła Trackpoint można umieścić blok <Extensions>, który pozwala na dołączanie dowolnych nowych danych.[7]

1.2.4 FIT

Bibliografia

[1] Ulrich Drepper, *What Every Programmer Should Know About Memory*, Red Hat, Inc., 2007

[2] Agner Fog, *Optimizing software in C++: An optimization guide for Windows, Linux, and Mac platforms*, Agner Fog, 2025

[3] John L. Hennessy, J. L., David A. Patterson, *Computer Architecture: A Quantitative Approach Fifth Edition*, Morgan Kaufmann, 2017

[4] Richard Acton, *Data-Oriented Design: Software engineering for limited resources and short schedules*, Richard Acton, 2018

[5] Bernhard Hofmann-Wellenhof, Herbert Lichtenegger, Elmar Wasle, *GNSS – Global Navigation Satellite Systems: GPS, GLONASS, Galileo, and more*, SpringerWien-NewYork, Austria, 2008

[6] TopoGrafix, *GPX: the GPS Exchange Format*, https://www.topografix.com/gpx.asp, [dostęp: 14.04.2026]

[7] Garmin, *Training Center Database XML Schema*, https://www8.garmin.com/xmlschemas/TrainingCenterDatabasev2.xsd, [dostęp: 14.04.2026]